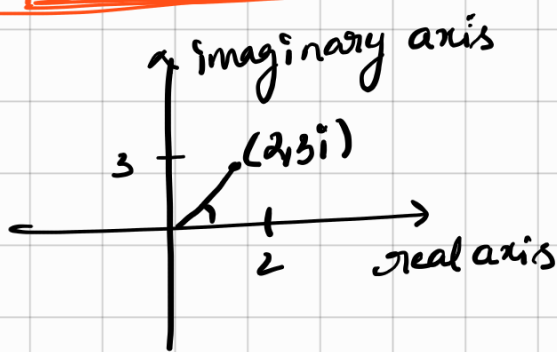


Sine Waves:

formula: $A \sin(2\pi ft + \theta)$

$\theta \rightarrow 0$ - sine $\theta \rightarrow \frac{\pi}{2}$ - cosine $\rightarrow$ orthogonal to each other

Complex Numbers:



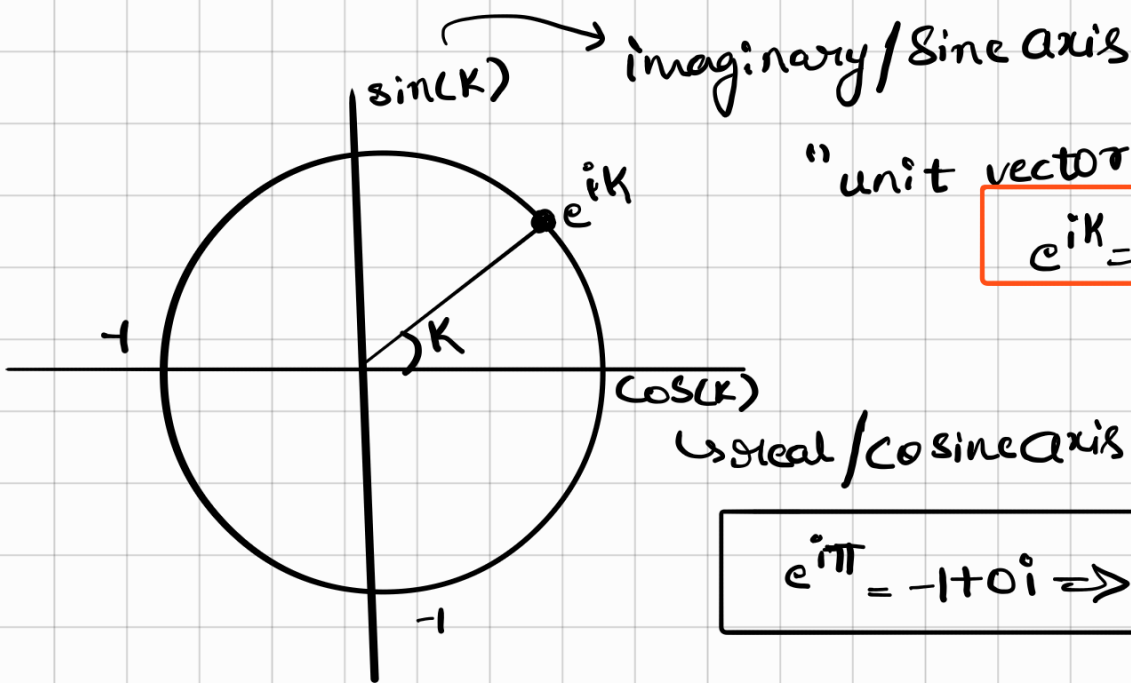
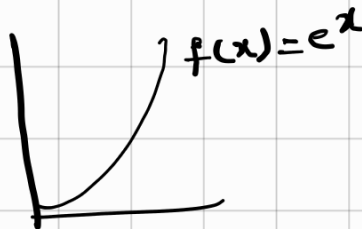
$$i = \sqrt{-1}$$

complex conjugate $\rightarrow$ mirror image flipped across real axis

Result of Fourier transform is Fourier coefficients $\rightarrow$ one Fourier coefficient per frequency and when we multiply each complex valued Fourier coefficient by its complex conjugate, then we will get power at that frequency

Euler's formula:

$$e = 2.718$$



"unit vector"

$$e^{iK} = \cos(K) + i\sin(K)$$

$\hookrightarrow$ can be solved, using Taylor's expansion

$$e^{i\pi} = -1 + 0i \Rightarrow e^{i\pi} + 1 = 0$$

$$me^{ik} = m(\cos(k) + i\sin(k))$$

→ we can use this notation to represent a fourier coefficient

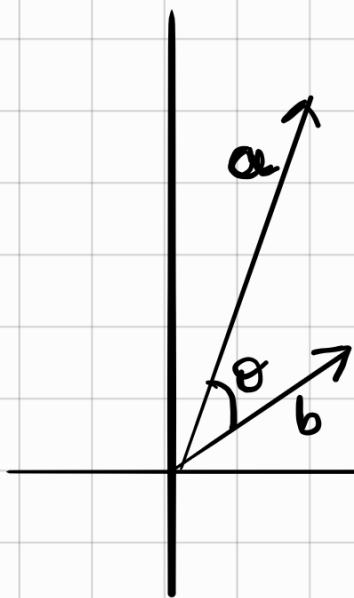
me^{ik} → phase of the fourier coefficient

→ Amplitude of the fourier coefficient

once we have fourier coefficients for lots of different frequencies then separate out m's here and that gives us amplitude spectrum, we can also square to get power spectrum and for k's we get phase spectrum

The Dot product:

$$d = a \cdot b = \langle a, b \rangle = a^T b = \sum_{i=1}^n a_i b_i$$



$$d = |a||b|\cos(\theta_{ab})$$

$$\theta < 90^\circ \rightarrow \cos(\theta) > 0 \Rightarrow d > 0$$

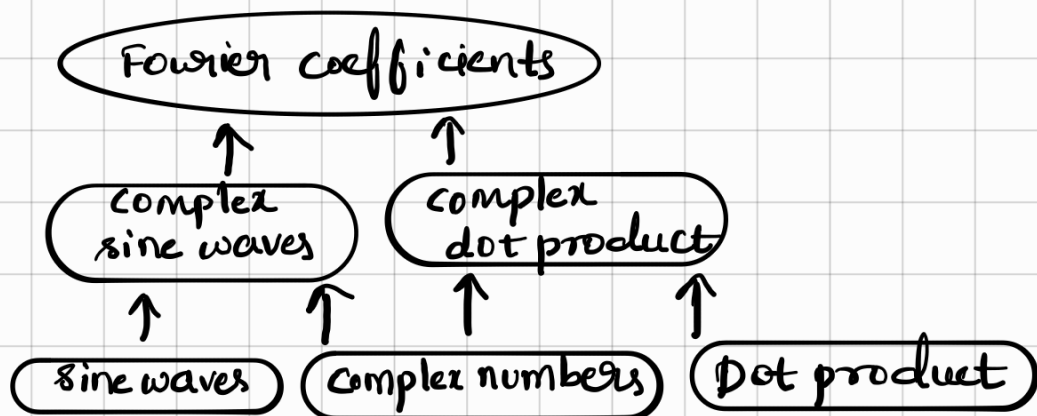
$$\theta > 90^\circ \rightarrow \cos(\theta) < 0 \Rightarrow d < 0$$

$$\theta = 90^\circ \rightarrow \cos(\theta) = 0 \Rightarrow d = 0$$

$$\theta = 0^\circ \rightarrow \cos(\theta) = 1 \Rightarrow d = |a||b|$$

orthogonal
collinear

Complex Sine Waves:



$e^{ik} = \cos(k) + i\sin(k)$ → Here k is a single number

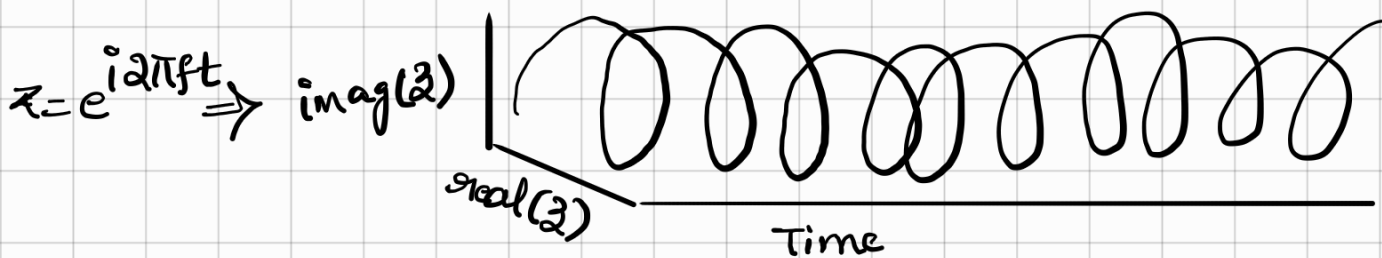
$$k = 2\pi f t + \theta$$

→ Here k is a vector of numbers because T is a vector of time points

$$\text{Complex sine wave} = e^{i(2\pi f t + \theta)}$$

Complex Sine wave → taking two dimensional axis (time on x and amplitude on y) and plotting both real parts ($\cos(2\pi f t)$) and imaginary part ($\sin(2\pi f t)$) together overlaid on same plot

is a 3 dimensional → Time on x axis
real part on y axis and
imaginary part on z axis



Complex Dot Product:

$$\begin{bmatrix} 1+4i \\ 0+i \\ 2-0i \end{bmatrix}^T \begin{bmatrix} 2 \\ 8 \\ 6 \end{bmatrix}$$

$$\begin{aligned} v^T w &= (2 \cdot 8i) + (8i) + (12) \\ &= 14 + 16i \end{aligned}$$

information from real value dot product:

→ mag → distance from 0
sign → left / right of 0

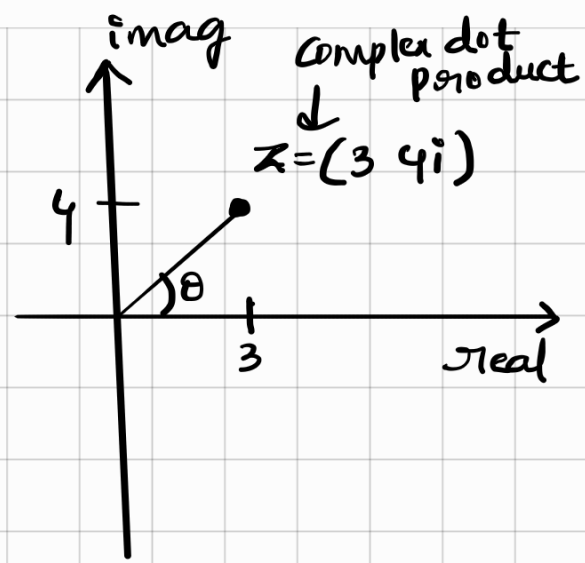
information from complex dot product:

$$|z| = (\text{real}(z)^2 + \text{imag}(z)^2)^{1/2}$$

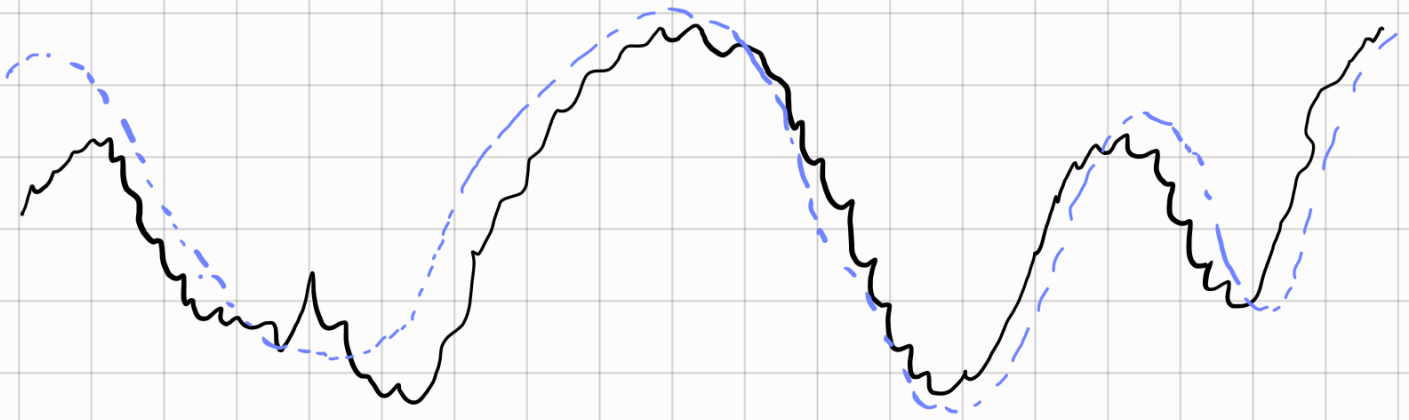
$$|z| = (z \times \bar{z})^{1/2}$$

$$\text{phase} \Rightarrow \tan(\theta) = \frac{\text{imag}(z)}{\text{real}(z)}$$

$$\theta = \text{atan}\left(\frac{\text{imag}(z)}{\text{real}(z)}\right)$$



Fourier coefficients:



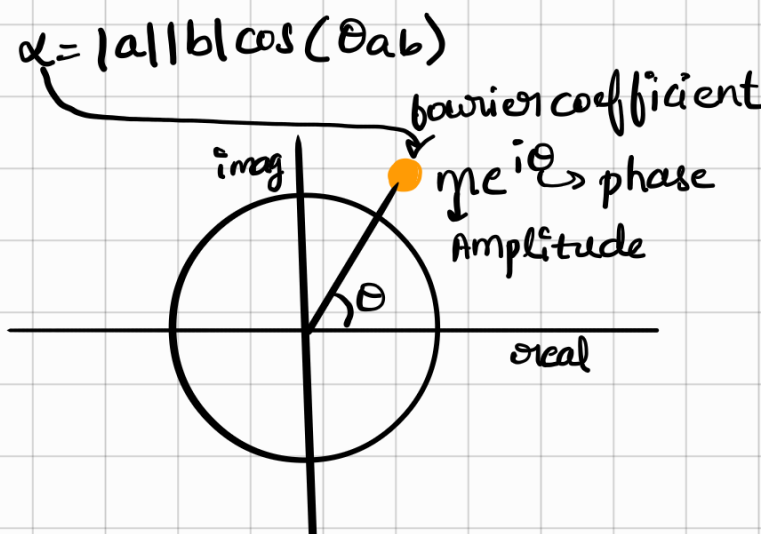
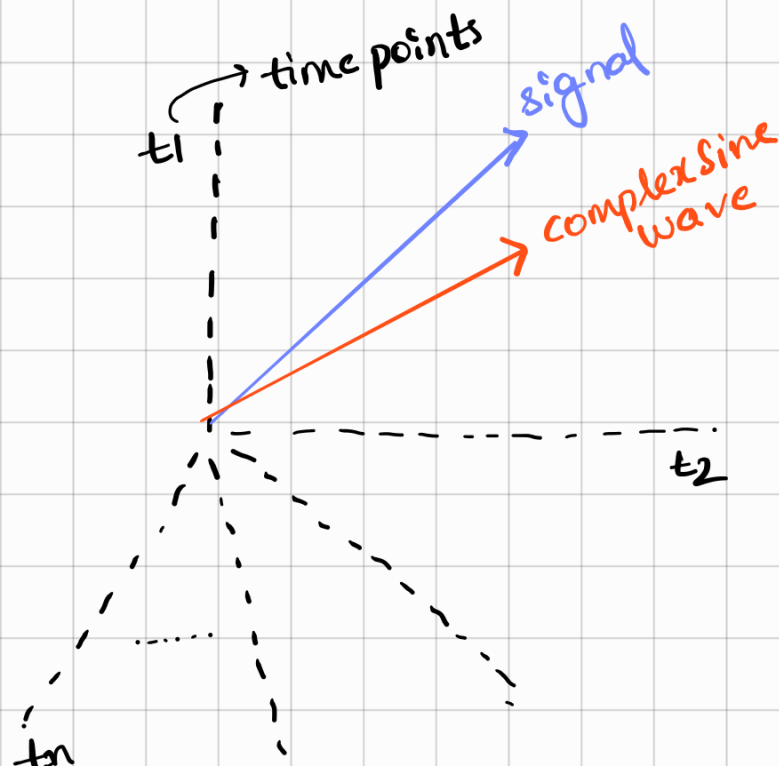
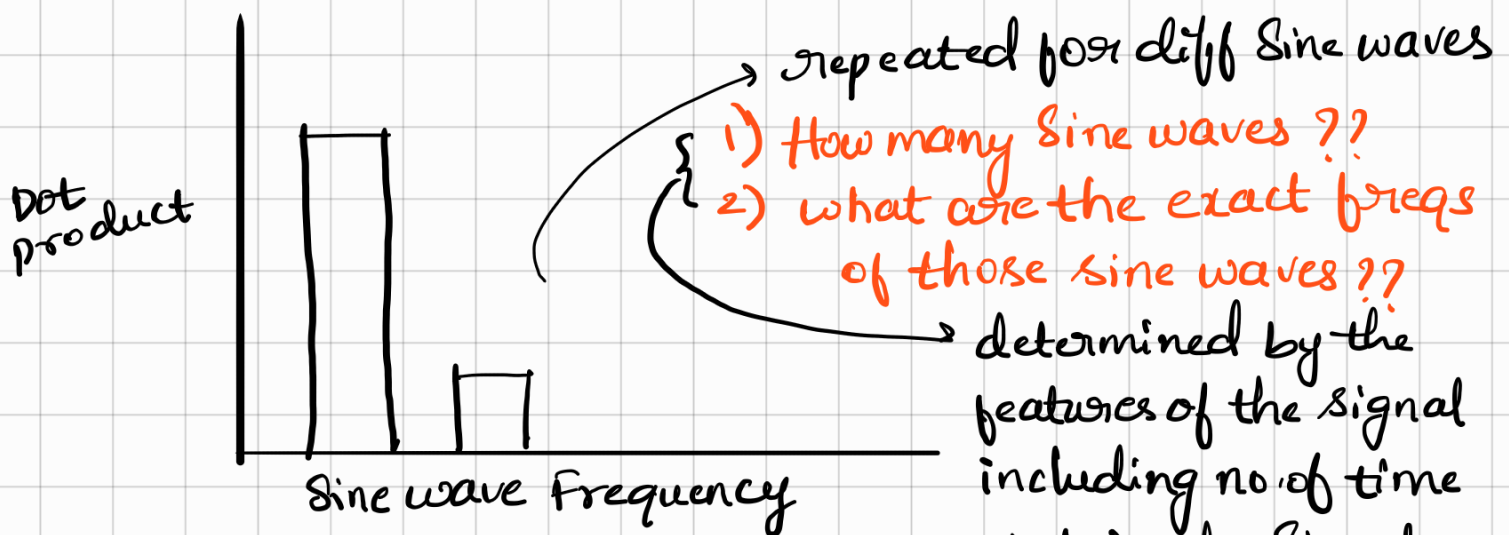
we will take sine wave at a particular frequency and line up the sine wave with signal and then compute the dot product between sine wave and signal

- Implies 2 things
- 1) sine wave have to be discretized
 - ↳ sine wave has to be sampled at same sampling rate as signal
 - 2) No. of time points in sine wave is same as no. of time points in signal → for dot product to be valid

Real value dot product of sine wave and signal is very much dependent on the phase of the signal with the sine wave that's why we should use complex valued sine wave and not real valued sine wave

↳ Accounts for any possible phase relationship b/w signal & the complex valued sine wave and that phase offset is handled because the signal is going to look something like cosine and something like sine

↳ two components of dot product, one corresponding to cosine & other to sine



How the DTFT works (loop algorithm):

loop over number of time points

create complex sine wave with length of signal, and frequency of looping index - 1

compute dot product between complex sine wave and signal

end

Amplitude spectrum is magnitude of fourier coefficients
phase spectrum is angle of fourier coefficients

*** Here No. of time points determines the no. of freqs that we can extract from the signal
in the signal

Frequencies in the fourier transform:

```
% loop over frequencies
for fi=1:N

    % create sine wave for this frequency
    fourierSin = exp( -1i*2*pi*(fi-1).*fourierTime );

    % compute dot product as sum of point-wise elements
    fourierCoefs(fi) = sum( fourierSin.* signal );

end
```

units are not in Hz. these are freqs in indices

Normalised time vector, goes from 0 to 1

$N \rightarrow$ no. of time points in the signal

because, we want fourier transform to be a general operation that always works, so we don't want to have a different fourier transform when we sample our data in milli seconds and a diff fourier transform when we sample our data in days

we want our fourier transform to work all the time regardless of the units of our original signal $\rightarrow$ that's why it is important that time vector is normalized and frequencies given in indices and not in some physically meaningful units like Hz.

Converting freqs to Hz:

$h3 = \text{linspace}(0, \text{sr}/2, N/2+1);$
 $\downarrow$
 vector of Hertz

$\rightarrow$ Nyquist freq

$\rightarrow$ linearly spaced numbers from 0 to $\text{sr}/2$ in $\frac{N}{2}+1$ steps

No. of frequencies b/w 0 and Nyquist is $\frac{N}{2}+1$, where N is the no. of time points in the signal

so, freq resolution is determined by the length of the signal

lower bound & upper bound of the frequencies that we can reconstruct from fourier transform

① What is the lowest freq that we can think about extracting from a signal?

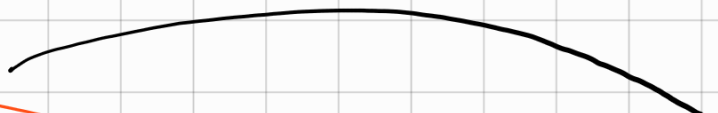
$f=2$



$f=1$



$f=0.5$



$f=0$

$\hookrightarrow$ it's a flat line with value of 1 for all points

"DC"
 (Direct current)
 $\hookrightarrow$ average of all values in signal

$e^0 \rightarrow$ vector of 1's
 then we multiply this with Signal and do sum
 according to above DTFT algo
 then we divide that sum by no. of elements
 in signal $\rightarrow$ this is just avg of signal

*** "Why divided by n?"

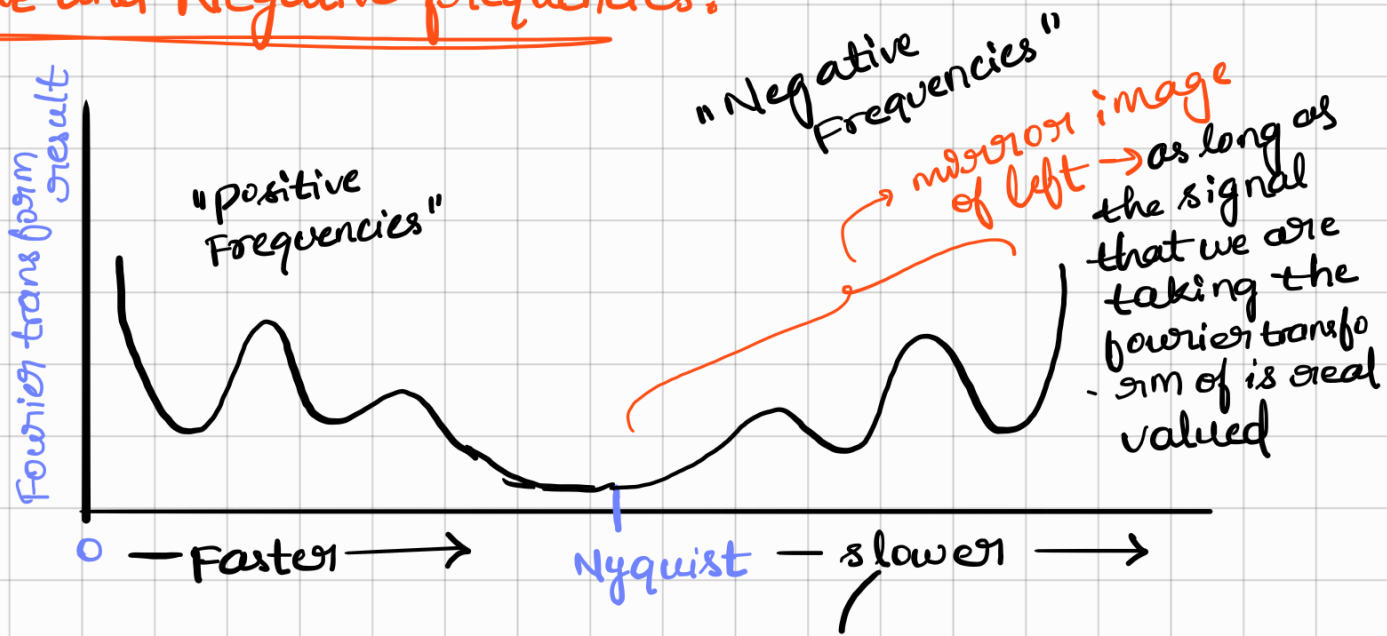
② what is the upper bound for freq that we can think about extracting from a signal?

we want to discretize and quantify signal, making sure that we can still recognize that its an oscillatory rhythmic signal
 minimum no. of points we need to sample from signal ??

Aliasing $\rightarrow$ artifact that is produced when we are sampling lower than the frequency

Nyquist freq $\rightarrow \frac{\text{sampling rate}}{2}$

positive and Negative frequencies:



↓
the reason for sine waves getting slower and slower is the sine waves themselves are spinning faster than how we can discretize them inside the fourier transform when we are creating them

→ sine wave frequencies are starting to get so fast that we cannot fully reconstruct the rhythmicity of sine wave

→ these freqs are getting faster and faster, but they are aliased to look slower and slower → these are called Negative Frequencies

←
Because they get aliased into slower freqs, we don't number these as being faster above the Nyquist, we start counting these in Hertz going back down towards zero and attaching a -ve sign to them

Zero Hz exists only one time and all other occurs 2 times

Amplitude of the signal is split evenly b/w the positive freqs and negative freqs

→ let's say in true signal, there is 10 microvolts at 10 Hz, the amplitude will show 5 at each left and corresponding right freq points

"Why they gets split?"

$$\cos(k) = \frac{e^{ik} + e^{-ik}}{2}$$

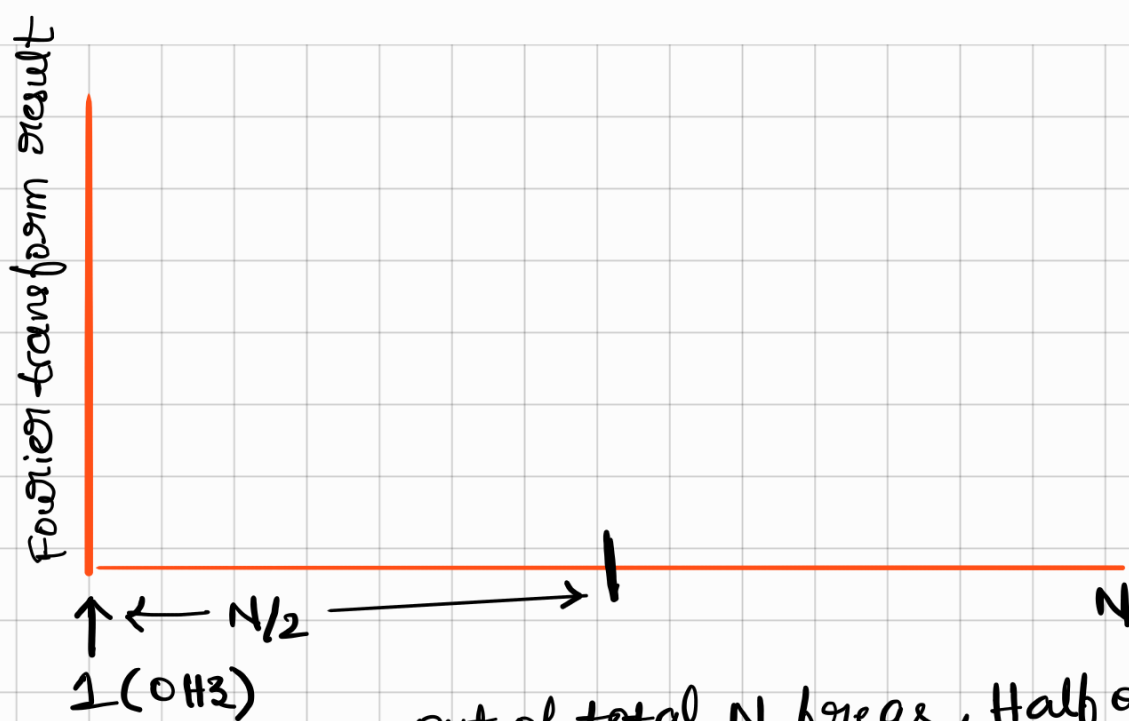
↓
real valued signal

complex valued sine wave

$$\cos(2\pi ft) = \frac{e^{i2\pi ft} + e^{-i2\pi ft}}{2}$$

→ corresponds to positive freqs

→ corresponds to negative freqs



out of total N freqs, Half of them are +ve, Half of them are -ve and then we have one extra because of 0Hz
 $\hookrightarrow \text{so } (N/2 + 1)$

Why we need exactly N freqs in fourier transform ?? $\longrightarrow$

why dont we stop at just $\frac{N}{2}$ freqs and stop the fourier transform at the Nyquist
 $\hookrightarrow$ because we need to make sure that fourier transform is a perfect invertible transform without any information loss, we dont want to approximate the signal, we want to quantify it exactly in the freq domain

`h3 = linspace(0, Srate, N);`

$\hookrightarrow$ this is also correct as long as we are not interpreting freqs above Nyquist

Accurate scaling of Fourier coefficients:

```
% loop over frequencies
for fi=1:N

    % create sine wave for this frequency
    fourierSin = exp( -1i*2*pi*(fi-1).*fourierTime );

    % compute dot product as sum of point-wise elements
    fourierCoefs(fi) = sum( fourierSin.* signal );

end
```

→ As the signal gets longer and longer, the Fourier coefficient for the zero Hz frequency gets larger and larger because we are summing together more & more numbers

→ to get avg value, we want to divide by no. of datapoints in the signal

→ same concept applies for every frequencies

To recover the accurate units (e.g., μV) from Fourier transform:

- 1) Divide Fourier coefficients by N
- 2) Add Negative to positive freq coefficients (double positive coefficients)

NOTE: 0 Hz and Nyquist frequencies are Not doubled, they have no corresponding negative frequency

NOTE: scaling changes the y-axis values, not the shape of the spectrum!
scaling might not be necessary

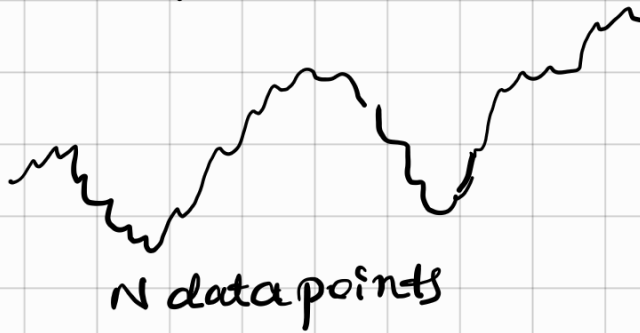
Fourier transform: time domain to freq domain

↳ lossless

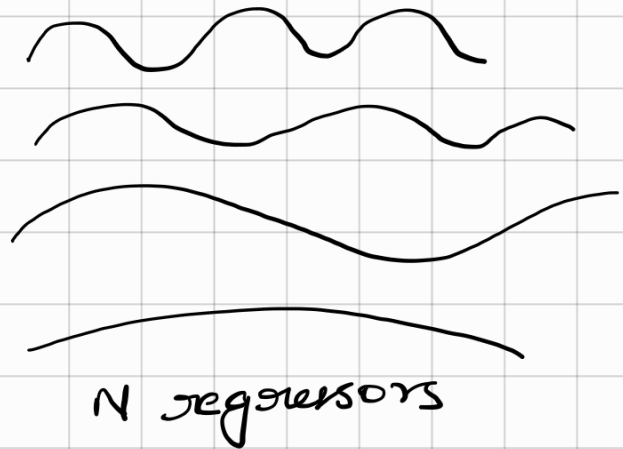
↳ it is Not an approximation of signal,
it is exact representation of signal

Statistics perspective:

Time domain signal
(dependent variable)



complex sine waves
(independent variables)

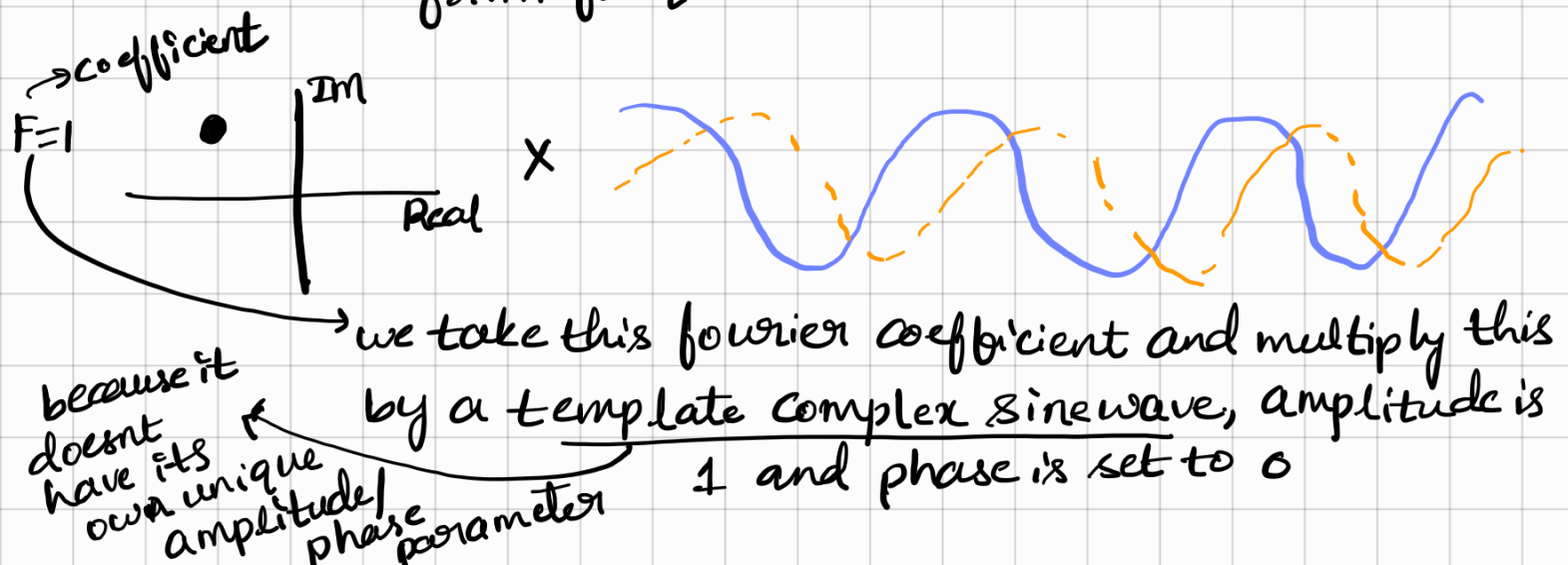


when we are trying to build multiple regression model, explaining variance of N data points based on N independent variables

0 degrees of freedom → because of same no. of predictors as no. of data points
100% variance explained

The Inverse Fourier transform:

from freq domain to time domain



$F=2$

$\vdots$

$F=N$

x for here freq of template complex sine wave will be 2

x

sum all modulated complex sine waves together
↳ this will be time domain signal

How the Inverse Fourier transform works?

loop over freqs

create complex sine wave with length of signal, and freq of time-point - 1

multiply the sine wave by the complex fourier coefficient

sum the "modulated complex sine waves"

end

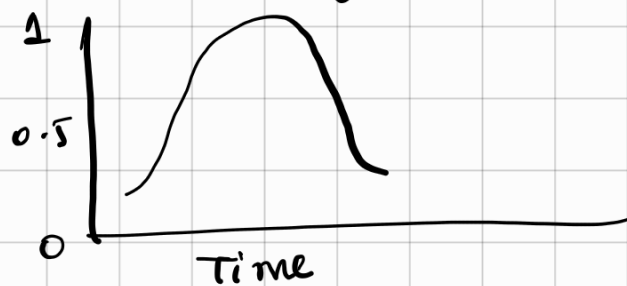
Divide by N (because the loop involved summing N sine waves)

Reasons for using forward / Inverse Fourier transform:

- 1) Filtering (spectral source separation)
- 2) Decrease computation time (convolution theorem)

Frequency resolution and Zero padding:

for increasing freq resolution, we will do zero padding at end of signal



Motivations for Zero padding:

- 1) obtain a specific freq in FFT (Spectral interpolation)
- 2) Match FFT lengths of convolution
- 3) smoother spectral plots

Signal Nonstationarities and their effects on the power Spectrum:

The results of the fourier transform are easily interpretable only for stationary signals

stationarity: A signal is stationary when its statistical characteristics do not significantly change over time

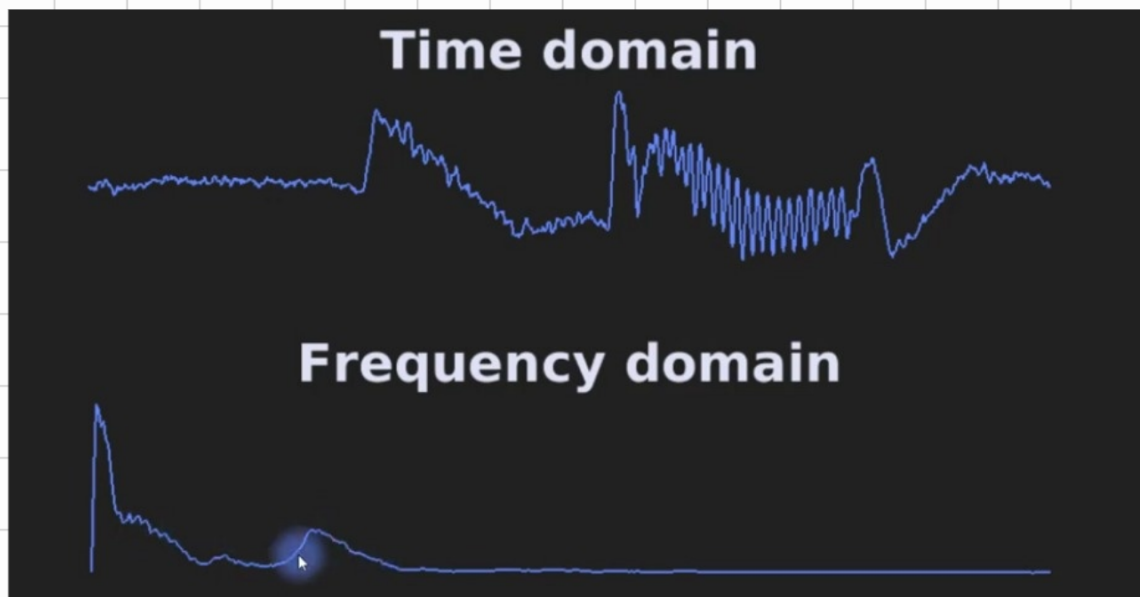
Non-stationarity: A signal is Non stationary when stationarity is violated

→ three sources of definitional ambiguities in this statement:

- 1) There are many features of signals
- 2) "significant" is threshold dependent
- 3) window-size and location is subjective

Limitations of "static" spectral analyses:

→ It is easily interpretable only for stationary signals



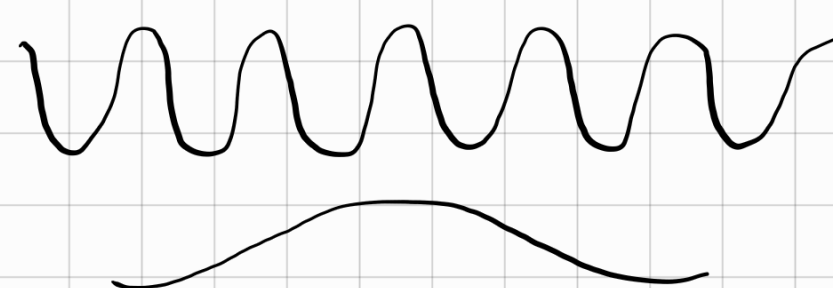
wavelet
convolution
↳ For
Time
Frequency
Analysis

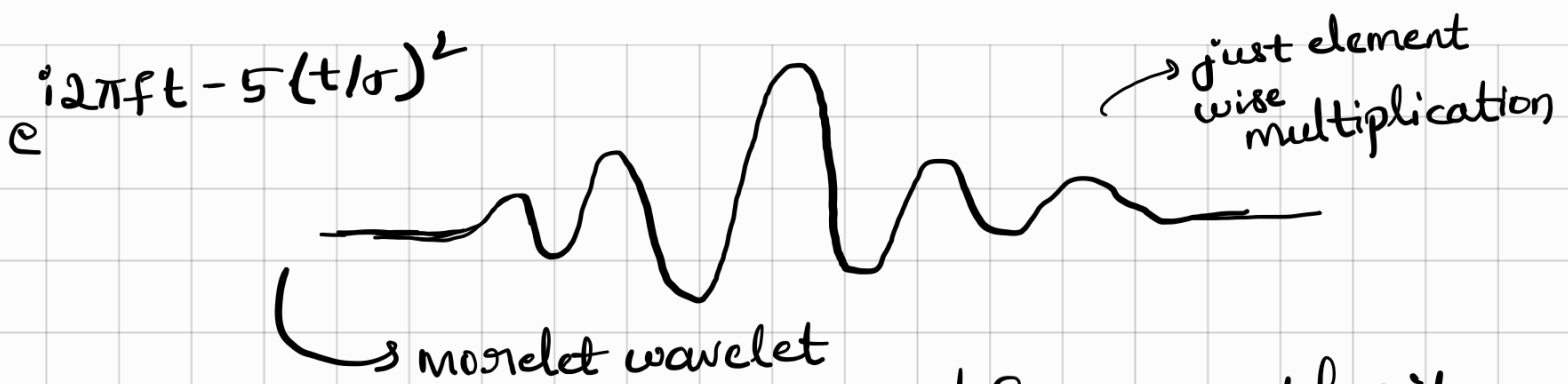
Key properties of wavelets:

- 1) should taper down to zero at the beginning and end
- 2) Integrate to zero → positives & negatives cancel

where do wavelets come from?

Pointwise multiply the sine wave with gaussian


$$e^{i2\pi ft}$$
$$e^{-.5(t/\sigma)^2}$$



If its a Complex valued Sine wave, then it is complex morlet wavelet

why wavelets provide temporal specificity?

sliding kernel along the signal and computing dot product over and over again

where as in fourier transform entire signal is weighted equally

